STATSM148_final_report

March 13, 2024

[1]:
```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import os
os.chdir('/Users/tsukasamiyaji/Desktop/Python3')
import xgboost as xgb
from xgboost import XGBClassifier, plot_importance
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import OrdinalEncoder
from collections import Counter
from kneed import KneeLocator
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from mlxtend.frequent_patterns import apriori as ap, association_rules as ap_rl
from icecream import ic
from joblib import Parallel,delayed
import shap
```

1 Data Preparation

[2]:
```
df = pd.read_csv('Fingerhut_modified.csv')
df = df.drop(['Unnamed: 0'],axis=1)
pivot_table = pd.pivot_table(df, values='count', index='account_id',␣
 ↪columns='ed_id',aggfunc='sum').fillna(0)
export_piv2 = pd.DataFrame(pivot_table)
export_piv2 = export_piv2.reset_index()
export_piv2.columns =␣
 ↪['account_id','1','2','3','4','5','6','7','8','9','10','11','12','13','14','15','16','17','
export_piv2.set_index('account_id',inplace=True)
export_piv2.head()
```

[2]:
```
                1    2    3     4    5    6    7    8    9   10  …    19  \
account_id                                                        …
-2147477843  4.0  1.0  0.0   9.0  5.0  2.0  1.0  2.0  0.0  0.0  …   0.0
-2147476504  3.0  1.0  1.0  18.0  8.0  7.0  1.0  1.0  0.0  0.0  …   8.0
-2147476077  0.0  1.0  1.0   8.0  0.0  0.0  0.0  0.0  0.0  0.0  …  10.0
-2147475397  0.0  1.0  0.0   0.0  0.0  0.0  0.0  0.0  0.0  0.0  …   0.0
-2147473858  2.0  0.0  2.0   8.0  3.0  1.0  0.0  0.0  0.0  0.0  …  18.0

               20   21   22   23   24   26   27   28   29
account_id
-2147477843  0.0  0.0  0.0  0.0  0.0  0.0  1.0  1.0  1.0
-2147476504  0.0  0.0  0.0  0.0  0.0  0.0  1.0  1.0  1.0
-2147476077  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
-2147475397  0.0  0.0  1.0  0.0  0.0  0.0  0.0  0.0  0.0
-2147473858  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0

[5 rows x 28 columns]
```

[3]:
```
cond1 = export_piv2['7'] > 0
cond2 = export_piv2['18'] > 0
cond3 = export_piv2['29'] > 0

condition_act  = [cond3, ~cond3]
choices_act    = [0,1]

condition_pur  = [(cond1 | cond2), (~cond1 | ~cond2)]
choices_pur     = [0,1]

export_piv2["condition_purchase"] = np.select(condition_pur, choices_pur,␣
 ↪default=np.nan)
export_piv2["condition_activation"] = np.select(condition_act, choices_act,␣
 ↪default=np.nan)
export_piv2.head()
```

[3]:
```
                1    2    3     4    5    6    7    8    9   10  …   21   22  \
account_id                                                        …
-2147477843  4.0  1.0  0.0   9.0  5.0  2.0  1.0  2.0  0.0  0.0  …  0.0  0.0
-2147476504  3.0  1.0  1.0  18.0  8.0  7.0  1.0  1.0  0.0  0.0  …  0.0  0.0
-2147476077  0.0  1.0  1.0   8.0  0.0  0.0  0.0  0.0  0.0  0.0  …  0.0  0.0
-2147475397  0.0  1.0  0.0   0.0  0.0  0.0  0.0  0.0  0.0  0.0  …  0.0  1.0
-2147473858  2.0  0.0  2.0   8.0  3.0  1.0  0.0  0.0  0.0  0.0  …  0.0  0.0

               23   24   26   27   28   29  condition_purchase  \
account_id
-2147477843  0.0  0.0  0.0  1.0  1.0  1.0                 0.0
-2147476504  0.0  0.0  0.0  1.0  1.0  1.0                 0.0
-2147476077  0.0  0.0  0.0  0.0  0.0  0.0                 1.0
```

```
-2147475397  0.0  0.0  0.0  0.0  0.0  0.0                 1.0
-2147473858  0.0  0.0  0.0  0.0  0.0  0.0                 1.0

             condition_activation
account_id
-2147477843                   0.0
-2147476504                   0.0
-2147476077                   1.0
-2147475397                   1.0
-2147473858                   1.0

[5 rows x 30 columns]
```

- In the data preparation phase, a pivot table is constructed where each cell represents the frequency of specific events for each customer account ID. For example, if a customer browses the website 20 times, this is recorded under event ID No.4 with a frequency of 20. Additionally, customers are categorized based on whether they made a purchase and whether they activated their account. A purchase is indicated by a customer undertaking event No.7 (placing an order on the website) or event No.18 (placing an order by phone). Account activation is indicated by event No.29 (account activation). These customer actions are then encoded in the pivot table, with '0' signifying an action taken (purchase made/account activated) and '1' indicating the opposite (no purchase made/account not activated), creating a comprehensive matrix that combines the frequency of event occurrences with the binary classifications of purchase and activation status.

2 XGBoost (Purchase)

[4]:
```
ids_to_remove = ['7','18','28']
export_piv2_1 = export_piv2.drop(columns=ids_to_remove)
```

- Before applying XGBoost for classification, it's crucial to eliminate certain events that directly indicate a purchase, to ensure the model does not use these as direct predictors of purchasing behavior. Specifically, event IDs No.7 and No.18, which denote product purchases via the website and by phone, respectively, are removed to prevent the model from relying on these clear indicators. Similarly, event No.28, related to order shipment, is also excluded because it inherently signifies that a purchase has occurred. By omitting these event IDs, the objective is to assess the impact of other, seemingly unrelated events on the likelihood of making a purchase, thereby focusing on more subtle indicators of customer behavior.

[5]:
```
# Modify the dependent variable as per your requirements
X, y = export_piv2_1.drop(['condition_purchase', 'condition_activation'],␣
 ↪axis=1), export_piv2_1[['condition_purchase']]

# Split into train and test data
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.
 ↪3,random_state=42)
```

```
# Construct dmatrix for train model
dtrain_clf = xgb.DMatrix(X_train, y_train, enable_categorical=True)
dtest_clf = xgb.DMatrix(X_test, y_test, enable_categorical=True)

# Train a model, modify the objective method as per your requirements
xgb_classifier = xgb.XGBClassifier(n_estimators=100, objective='binary:hinge',␣
 ↪tree_method='hist', eta=0.1, max_depth=10, enable_categorical=True)
xgb_classifier.fit(X_train, y_train)

# make predictions for test data
y_pred = xgb_classifier.predict(X_test)
xgb_prediction = [round(value) for value in y_pred]

# Print accuracy rate
print(f"Accuracy rate: {sum(xgb_prediction == y_test['condition_purchase'])/
 ↪len(xgb_prediction)}")

# Sample numeric vector from classification model
numeric_vector = xgb_prediction

# Count the occurrences of each element
element_counts = Counter(numeric_vector)

# Print the result
print("Number of Each Element:")
for element, count in element_counts.items():
    print(f"{element}: {count}")

# Sample numeric vector from actual test data
numeric_vector = y_test['condition_purchase']

# Count the occurrences of each element
element_counts = Counter(numeric_vector)

# Print the result
print("Number of Each Element:")
for element, count in element_counts.items():
    print(f"{element}: {count}")
```

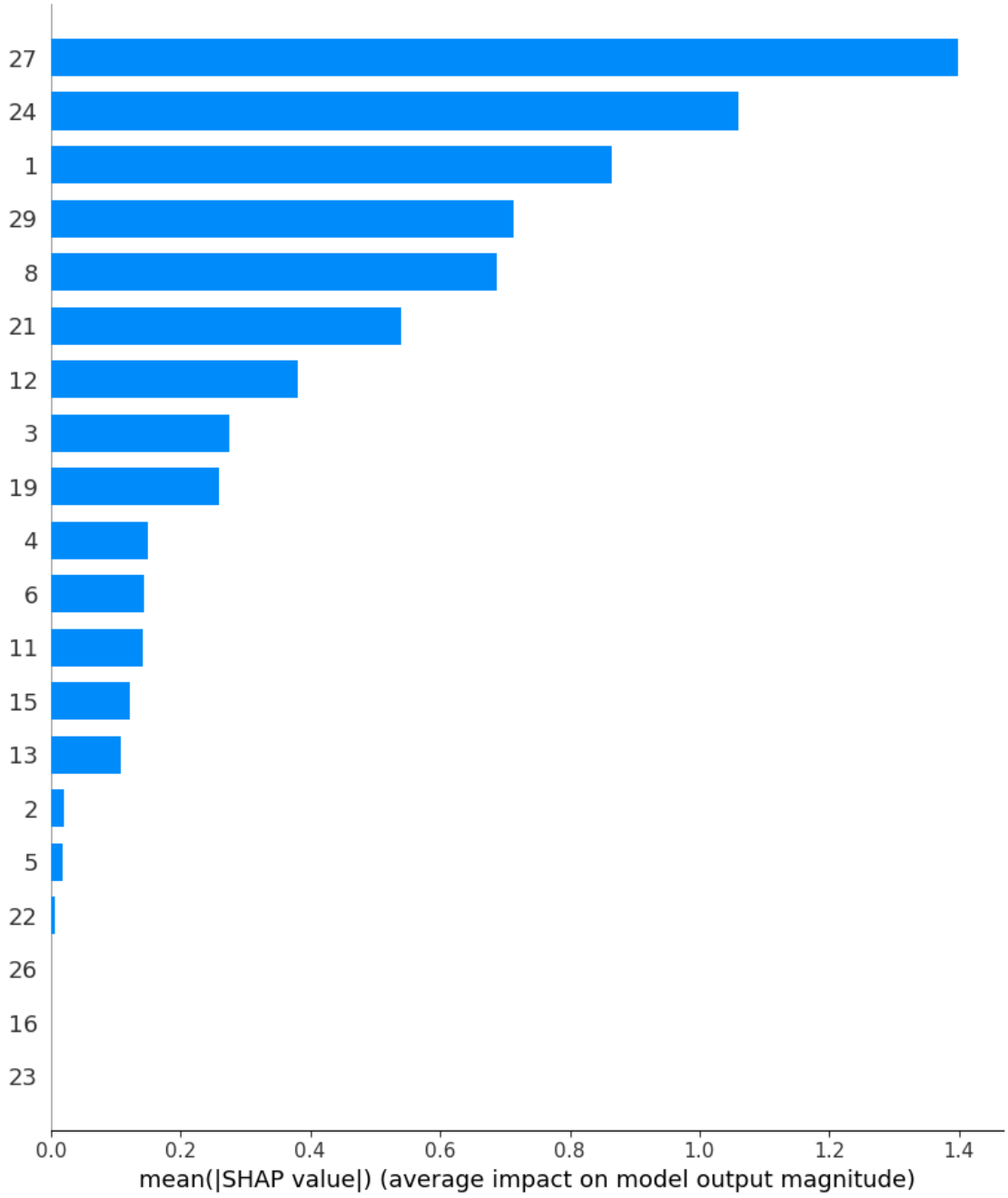
```
Accuracy rate: 0.9712487371263718
Number of Each Element:
1: 414464
0: 106174
Number of Each Element:
1.0: 405353
0.0: 115285
```

- After removing specific event IDs that directly affect classification, we trained the XGBoost

model using the training data, which was split into 70% for training and 30% for testing, randomly. In the XGBoost model, we set the objective as `binary:hinge` because the output is binary, and this setting is used when probabilities, rather than just class labels, are desired. Additionally, this objective achieved the highest accuracy rate. Our accuracy rate for the binary classification using 30% test data, determining whether customers purchase products or not, is 0.971, indicating a very high accuracy level. We also observed the number in each classification group, where 1 indicates non-purchase, and 0 indicates purchase. However, since XGBoost is a supervised learning algorithm for classification problems, it does not inherently identify influential features within the data. Therefore, we introduced SHapley Additive exPlanations (SHAP) values, conceived by Lloyd Shapley. SHAP values offer an extensive framework for interpreting machine learning models by quantifying each feature's contribution to individual predictions, thus significantly enhancing model transparency. They ensure fair attribution of the prediction output among all features, providing insightful observations both locally and globally about model behavior. Their consistent nature guarantees that a feature's contribution is accurately reflected, irrespective of model changes. Furthermore, SHAP values are model-agnostic, applicable across a diverse range of machine learning techniques from deep learning to tree-based and linear models, making them a versatile tool for understanding and elucidating model predictions. In contrast, traditional feature importance methods, widely used in complex machine learning contexts such as XGBoost and other ensemble techniques, often fall short. They typically fail to provide detailed insights at the individual prediction level, overlook feature interactions, and can be misleading due to sensitivity to correlated or high cardinality features. These methods also struggle with inconsistency, model specificity, capturing non-linear relationships, and a lack of standardization across different model types. In this scenario, SHAP values prove to be more reliable and accurate than traditional feature importance metrics in XGBoost packages, offering a deeper insight into our XGBoost model and identifying which events most significantly affect purchasing decisions.

[6]:
```
# Use SHAP to explain the output of the model
explainer = shap.TreeExplainer(xgb_classifier)
shap_values = explainer.shap_values(X_test)
```

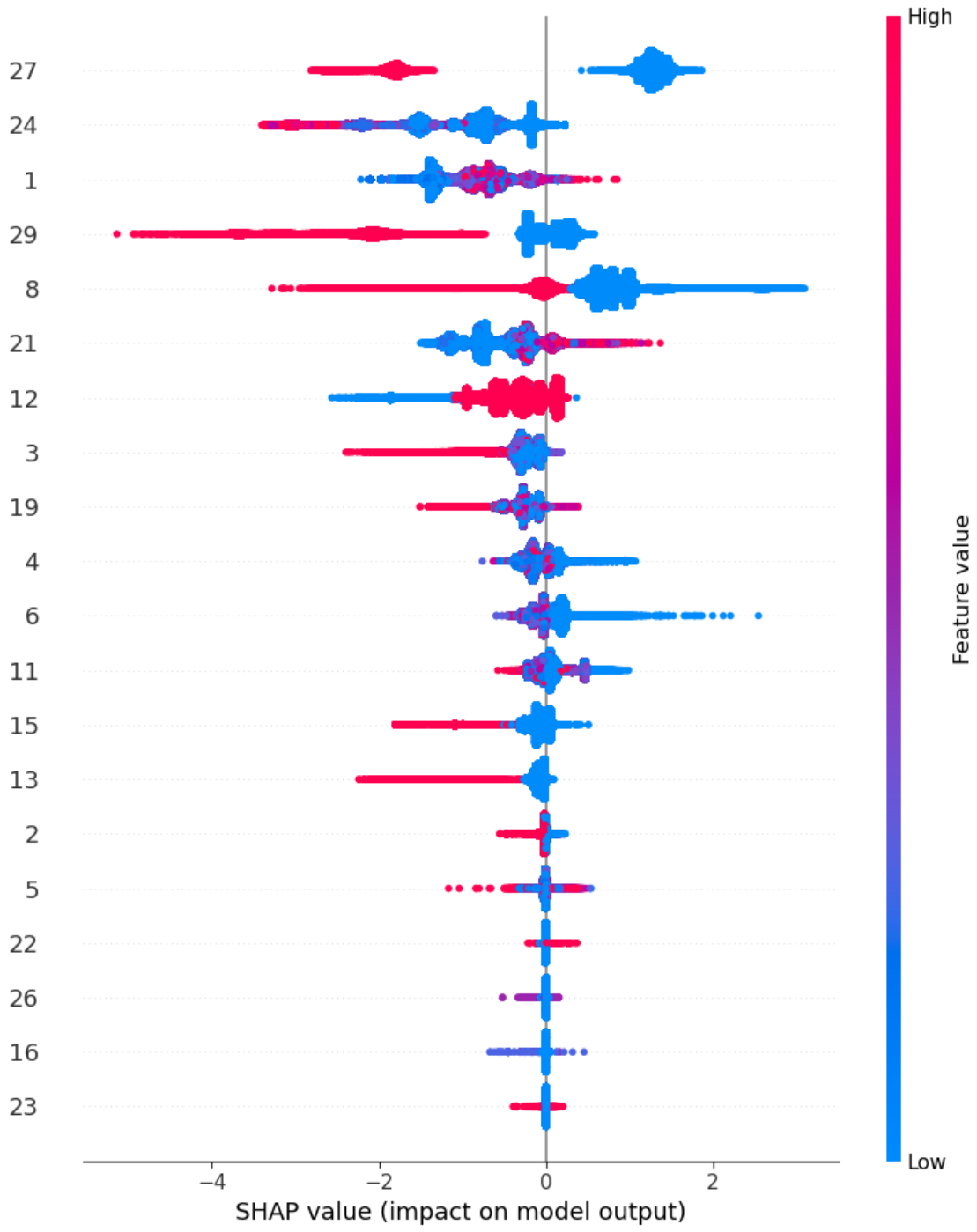
[7]:
```
# Summary plot of feature importance using SHAP
shap.summary_plot(shap_values, X_test.values, plot_type="bar", feature_names =␣
↪X_test.columns)
```



- Here is the feature importance derived from SHAP values. Higher values indicate a greater impact on our classification. The graph reveals that events No.27, No.24, No.1, No.29, and No.8 significantly influence classification. No.27, representing the clearance of downpayments made by customers, is crucial for determining purchase decisions, which makes sense logically. Similarly, No.24, indicating a click on a campaign email, plays a vital role in the purchasing decision. Event No.1, the creation of a promotion, suggests that providing promotion code can influence purchases. No.29, related to account activation, is another significant event,

underscoring its importance in the purchasing process. Additionally, No.8, which involves placing a downpayment, is highlighted as influential. While No.21, sending a catalog email to customers, is crucial, it does not contribute as significantly as the others. Campaign and catalog emails evidently wield substantial influence over purchasing decisions. Account activation and distribution of promotion codes also significantly impact purchasing. The downpayment event is noteworthy as well. However, while this plot clarifies which events influence purchasing decisions, it does not indicate whether their impact is positive or negative on the classification of purchasing. To discern the direction of these influences (positive or negative), further analysis with additional plots is necessary.

[8]:
```
shap.summary_plot(shap_values, X_test.values, feature_names = X_test.columns)
```



- The SHAP summary plot above illustrates the impact of different events on our binary classification model. The feature value here represents the frequency of each event for individual customers, essentially tracking how often each event occurs for them. For example, a red dot on the horizontal line for event 27 indicates customers with a high frequency of this event, which correlates with a negative SHAP value. This suggests that customers with more fre-

quent interaction in event 27, which is related to clearing downpayments, are more likely to make a further purchase (classified as 0). Conversely, a blue dot signifies customers with a low frequency of the event. When red dots are observed on the left side of the plot, it implies that a higher occurrence of the event is associated with an increased likelihood of purchasing. If red dots appear on the right side, it indicates the opposite: a higher occurrence corresponds with a decreased likelihood of purchase. Event 1, which involves sending promotion codes, presents a mixed picture; a limited number of promotions sent to a customer could lead to a purchase, but an excess may lose its effectiveness, possibly due to the perceived availability of discounts at any time, diminishing the urgency to buy. Event 24, involving customers clicking on campaign emails, clearly shows that those who engage more with campaign emails are more likely to purchase products than those who click on few or no campaign emails. Similarly, event 29, account activation, indicates that customers who activate their accounts are more inclined to purchase, with no significant impact observed for non-purchases. Event 8, related to placing downpayments, suggests that more frequent downpayments are indicative of a willingness to purchase, although it's not a definitive predictor since some high-frequency downpayments align with a SHAP value of 0, indicating no association with the outcome. Lastly, event 21, receiving catalog emails, seems to suggest that receiving an excessive number of catalog emails might dissuade purchases, while a moderate amount could encourage them, potentially due to over-saturation leading to annoyance.

[9]:
```
# Prepare for Apriori
xgb_pur_df = X_test.reset_index()
xgb_prediction = pd.DataFrame(xgb_prediction)
xgb_pur_df = pd.concat([xgb_pur_df,xgb_prediction],axis=1)
xgb_pur_df.rename(columns={0: 'prediction'}, inplace=True)
xgb_pur_df_0 = xgb_pur_df[xgb_pur_df['prediction']==0]
xgb_pur_df_1 = xgb_pur_df[xgb_pur_df['prediction']==1]
```

3 XGBoost (Activate)

[10]:
```
ids_to_remove = ['12','15','29']
export_piv2_2 = export_piv2.drop(columns=ids_to_remove)
```

- In this analysis, we turn our attention to account activation, a step that occurs after purchasing but before the placement and clearance of downpayments, according to another set of data. Events 12, 15, and 29 have a direct association with account activation. To delve deeper into how other, less directly related events influence account activation, we have removed these particular events from our dataset. This allows us to explore the effects of other customer interactions on the likelihood of activating an account.

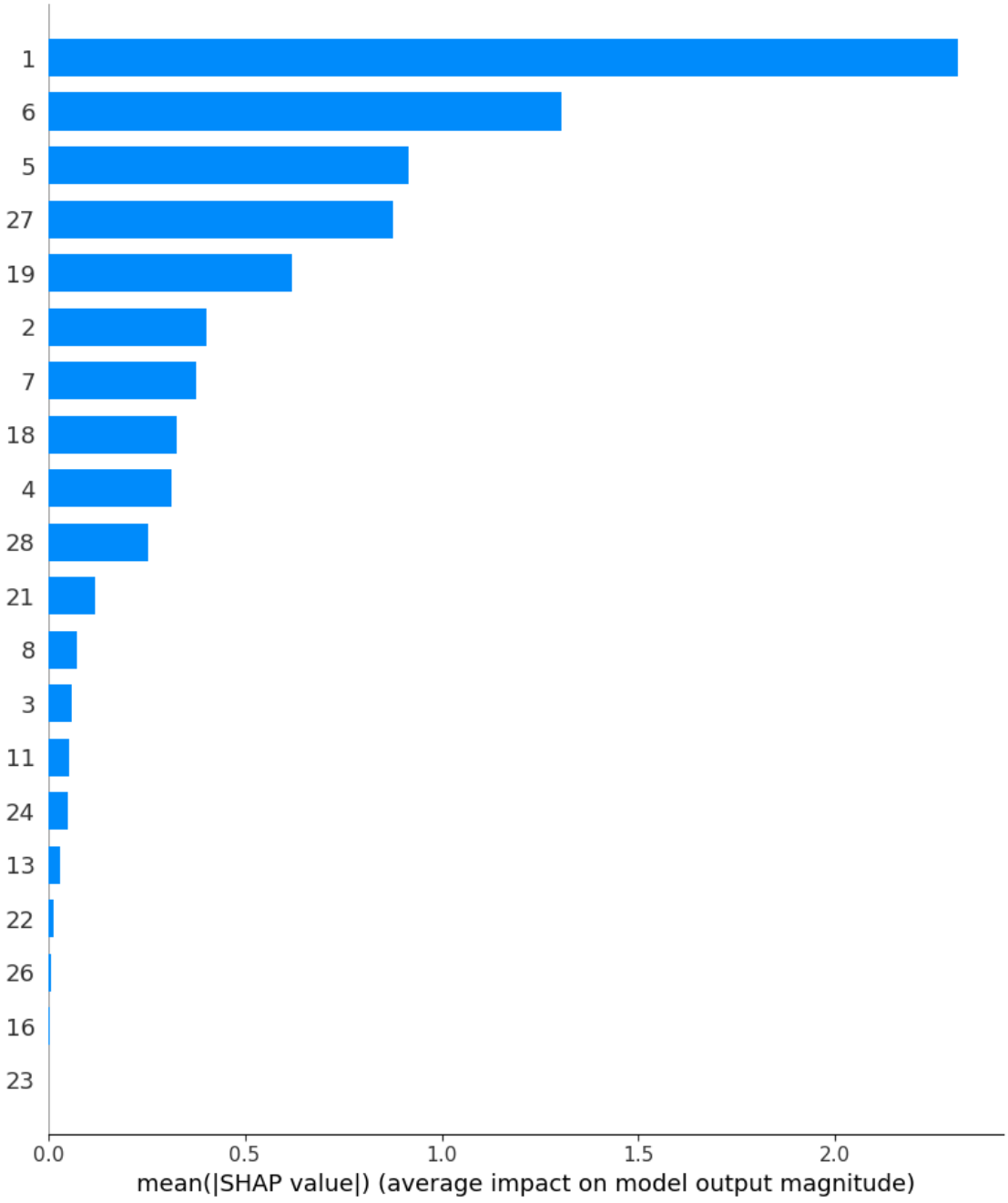
[19]:
```
# Modify the dependent variable as per your requirements
X, y = export_piv2_2.drop(['condition_purchase', 'condition_activation'],␣
 ↪axis=1), export_piv2_2[['condition_activation']]

# Split into train and test data
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.
 ↪3,random_state=42)
```

```
# Construct dmatrix for train model
dtrain_clf = xgb.DMatrix(X_train, y_train, enable_categorical=True)
dtest_clf = xgb.DMatrix(X_test, y_test, enable_categorical=True)

# Train a model, modify the objective method as per your requirements
xgb_classifier = xgb.XGBClassifier(n_estimators=100, objective='binary:hinge',␣
 ↪tree_method='hist', eta=0.1, max_depth=10, enable_categorical=True)
xgb_classifier.fit(X_train, y_train)

# make predictions for test data
y_pred = xgb_classifier.predict(X_test)
xgb_prediction = [round(value) for value in y_pred]

# Accuracy rate, modify the dependent variable as per your requirements
print(f"Accuracy rate: {sum(xgb_prediction == y_test['condition_activation'])/
 ↪len(xgb_prediction)}")

# Sample numeric vector
numeric_vector = xgb_prediction

# Count the occurrences of each element
element_counts = Counter(numeric_vector)

# Print the result
print("Number of Each Element:")
for element, count in element_counts.items():
    print(f"{element}: {count}")

# Sample numeric vector, modify the objective method as per your requirements
numeric_vector = y_test['condition_activation']

# Count the occurrences of each element
element_counts = Counter(numeric_vector)

# Print the result
print("Number of Each Element:")
for element, count in element_counts.items():
    print(f"{element}: {count}")
```

```
Accuracy rate: 0.9695028023309862
Number of Each Element:
1: 410017
0: 110621
Number of Each Element:
1.0: 395667
0.0: 124971
```

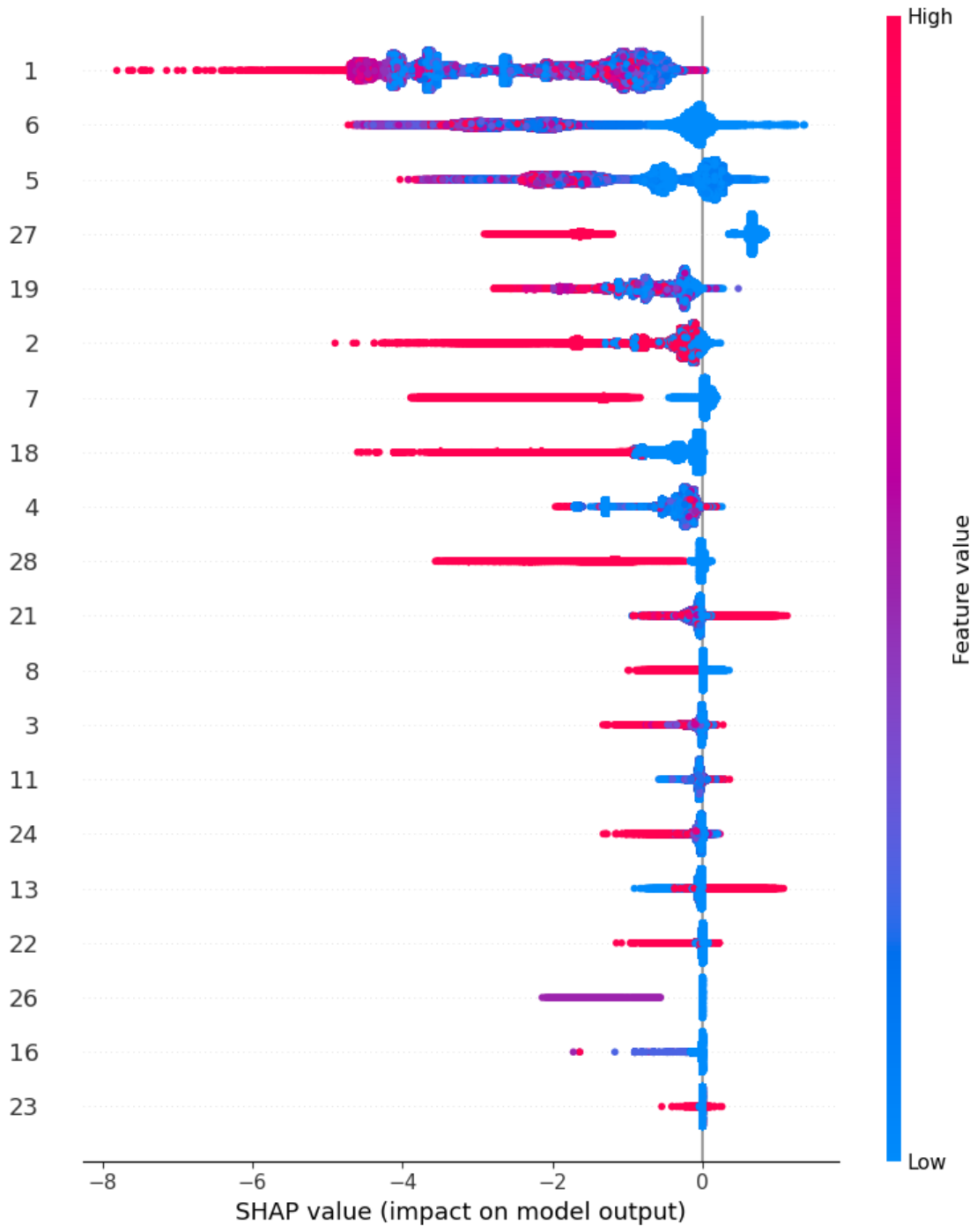
[20]:
```
# Use SHAP to explain the output of the model
explainer = shap.TreeExplainer(xgb_classifier)
shap_values = explainer.shap_values(X_test)
```

[21]:
```
# Summary plot of feature importance using SHAP
shap.summary_plot(shap_values, X_test.values, plot_type="bar", feature_names =␣
↪X_test.columns)
```



- This analysis identifies that event No.1, the creation of a promotion, has the most significant impact on classifying accounts as activated or not activated. Additionally, events No.6 and No.5, which correspond to beginning checkout and viewing the cart on the website, respectively, also play crucial roles in predicting account activation. Events No.27 and No.19, related to downpayment clearance and application views on the web, are influential as well. However, to discern whether these events impact account activation positively or negatively, a mere bar plot is insufficient. Therefore, we must refer to the SHAP summary plot presented below, which provides deeper insight into the directionality of each event's influence on the classification outcome.

[22]:
```
shap.summary_plot(shap_values, X_test.values, feature_names = X_test.columns)
```



- From above shap summary plot, No.1 has biggest impact to the classification.

[18]:
```
# Pripare for Apriori
xgb_act_df = X_test.reset_index()
xgb_prediction = pd.DataFrame(xgb_prediction)
```

```
xgb_act_df = pd.concat([xgb_act_df,xgb_prediction],axis=1)
xgb_act_df.rename(columns={0: 'prediction'}, inplace=True)
xgb_act_df_0 = xgb_act_df[xgb_act_df['prediction']==0]
xgb_act_df_1 = xgb_act_df[xgb_act_df['prediction']==1]
```

4 Apriori preparation

[17]:
```
exclude_column = ['account_id']
export_apr = export_piv2.apply(lambda x: x if x.name == exclude_column else (x␣
 ↪> 0).astype(int), axis=0)
remove =␣
 ↪['condition_purchase',        'condition_activation',        'condition_both']
export_apr = export_apr.drop(columns=remove)
export_apr.head()
```

[17]:

	1	2	3	4	5	6	7	8	9	10	…	19	20	21	22	23	24	26	\
account_id											…								
-2147477843	1	1	0	1	1	1	1	1	0	0	…	0	0	0	0	0	0	0	
-2147476504	1	1	1	1	1	1	1	1	0	0	…	1	0	0	0	0	0	0	
-2147476077	0	1	1	1	0	0	0	0	0	0	…	1	0	0	0	0	0	0	
-2147475397	0	1	0	0	0	0	0	0	0	0	…	0	0	0	1	0	0	0	
-2147473858	1	0	1	1	1	1	0	0	0	0	…	1	0	0	0	0	0	0	

	27	28	29
account_id			
-2147477843	1	1	1
-2147476504	1	1	1
-2147476077	0	0	0
-2147475397	0	0	0
-2147473858	0	0	0

[5 rows x 28 columns]

[30]:
```
xgb_pur_df_00 = xgb_pur_df_0.drop(['prediction'],axis=1)
xgb_pur_df_11 = xgb_pur_df_1.drop(['prediction'],axis=1)
xgb_act_df_00 = xgb_act_df_0.drop(['prediction'],axis=1)
xgb_act_df_11 = xgb_act_df_1.drop(['prediction'],axis=1)

ap_pur_df_0 = pd.merge(xgb_pur_df_00['account_id'], export_apr,␣
 ↪on='account_id', how='left')
ap_pur_df_1 = pd.merge(xgb_pur_df_11['account_id'], export_apr,␣
 ↪on='account_id', how='left')
ap_act_df_0 = pd.merge(xgb_act_df_00['account_id'], export_apr,␣
 ↪on='account_id', how='left')
ap_act_df_1 = pd.merge(xgb_act_df_11['account_id'], export_apr,␣
 ↪on='account_id', how='left')
```

```
ap_pur_df_00 = ap_pur_df_0.drop(['account_id'],axis=1)
ap_pur_df_11 = ap_pur_df_1.drop(['account_id'],axis=1)
ap_act_df_00 = ap_act_df_0.drop(['account_id'],axis=1)
ap_act_df_11 = ap_act_df_1.drop(['account_id'],axis=1)
```

5 Apriori (purchase)

[31]:
```
# support - % of how often each item appears
supp_thresh = 0.09

# lift
lift_thresh = 0.9

# FILTER THRESHOLD FOR # OF ITEMS IN SET (threshold INCLUDES the given value)
n_item_thresh = 3

# sort by columns
filter_cols = ["confidence",'support','lift']
```

[32]:
```
ids_to_remove = ['7','18','28']
ap_pur_df_00 = ap_pur_df_00.drop(columns=ids_to_remove)
ap_pur_df_11 = ap_pur_df_11.drop(columns=ids_to_remove)
```

[33]:
```
# APRIORI ALGORITHM (group 0)

ap_set = ap(ap_pur_df_00, min_support=supp_thresh, use_colnames=True)
rules = ap_rl(ap_set, metric = "confidence", min_threshold= lift_thresh)

# vectorize length func
v_len = np.vectorize(len)

# use it for condition in rule filtering
cond_0h = v_len(rules.antecedents.values) >= n_item_thresh
cond_0t = v_len(rules.consequents.values) >= n_item_thresh

# Apply both conditions to filter rules
filtered_rules = rules[cond_0h & cond_0t]

# Sort filtered rules by multiple columns for more insightful results
sorted_rules = filtered_rules.sort_values(by=filter_cols, ascending=[False,␣
 ↪False, False])

# Display top 10 meaningful and impactful rules
top_rules = sorted_rules.head(10)
top_rules
```

```
/opt/anaconda3/lib/python3.11/site-
packages/mlxtend/frequent_patterns/fpcommon.py:109: DeprecationWarning:
DataFrames with non-bool types result in worse computationalperformance and
their support might be discontinued in the future.Please use a DataFrame with
bool type
  warnings.warn(
```

[33]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage	conviction	zhangs_metric
160857	(24, 11, 27, 3, 1, 8)	(5, 29, 6)	0.098960	0.931499	0.098941	0.999810	1.073334	0.006760	359.868758	0.075827
189713	(24, 4, 11, 27, 3, 1, 8)	(5, 29, 6)	0.098923	0.931499	0.098904	0.999810	1.073334	0.006757	359.731756	0.075824
192688	(24, 11, 27, 3, 1, 12, 8)	(5, 29, 6)	0.098800	0.931499	0.098781	0.999809	1.073334	0.006749	359.286501	0.075814
203083	(24, 4, 11, 27, 3, 1, 12, 8)	(5, 29, 6)	0.098762	0.931499	0.098744	0.999809	1.073333	0.006746	359.149500	0.075810
192769	(19, 24, 11, 27, 3, 1, 8)	(5, 29, 6)	0.093808	0.931499	0.093789	0.999799	1.073323	0.006407	341.133799	0.075385
203260	(19, 24, 4, 11, 27, 3, 1, 8)	(5, 29, 6)	0.093780	0.931499	0.093761	0.999799	1.073323	0.006405	341.031048	0.075383
203970	(19, 24, 11, 27, 3, 1, 12, 8)	(5, 29, 6)	0.093648	0.931499	0.093629	0.999799	1.073322	0.006396	340.551543	0.075372
205534	(19, 24, 4, 11, 27, 3, 1, 12, 8)	(5, 29, 6)	0.093620	0.931499	0.093601	0.999799	1.073322	0.006394	340.448792	0.075369
153976	(24, 2, 11, 27, 1, 8)	(5, 29, 6)	0.146844	0.931499	0.146797	0.999679	1.073194	0.010012	213.599079	0.079941
186523	(24, 2, 4, 11, 27, 1, 8)	(5, 29, 6)	0.146787	0.931499	0.146740	0.999679	1.073194	0.010008	213.516878	0.079935

[34]:
```
# APRIORI ALGORITHM (group 1)

ap_set = ap(ap_pur_df_11, min_support=supp_thresh, use_colnames=True)
rules = ap_rl(ap_set, metric = "confidence", min_threshold= lift_thresh)
```

```
# vectorize length func
v_len = np.vectorize(len)

# use it for condition in rule filtering
cond_1h = v_len(rules.antecedents.values) >= n_item_thresh
cond_1t = v_len(rules.consequents.values) >= n_item_thresh

# Apply both conditions to filter rules
filtered_rules = rules[cond_1h & cond_1t]

# Sort filtered rules by multiple columns for more insightful results
sorted_rules = filtered_rules.sort_values(by=filter_cols, ascending=[False,␣
 ↪False, False])

# Display top 10 meaningful and impactful rules
top_rules = sorted_rules.head(10)
top_rules
```

```
/opt/anaconda3/lib/python3.11/site-
packages/mlxtend/frequent_patterns/fpcommon.py:109: DeprecationWarning:
DataFrames with non-bool types result in worse computationalperformance and
their support might be discontinued in the future.Please use a DataFrame with
bool type
  warnings.warn(
```

[34]:

	antecedents	consequents	antecedent support	consequent support	\
3820	(11, 5, 24, 21)	(1, 12, 4)	0.097902	0.623355	
2817	(11, 24, 21)	(1, 12, 4)	0.106991	0.623355	
3497	(11, 2, 21, 6)	(1, 12, 4)	0.099681	0.623355	
3887	(11, 19, 2, 6)	(5, 12, 4)	0.109648	0.464359	
4011	(2, 21, 5, 11, 6)	(1, 12, 4)	0.099492	0.623355	
3985	(19, 2, 11, 6, 1)	(5, 12, 4)	0.104945	0.464359	
3839	(11, 3, 2, 6)	(5, 12, 4)	0.092001	0.464359	
3443	(11, 5, 2, 21)	(1, 12, 4)	0.127878	0.623355	
2227	(11, 2, 21)	(1, 12, 4)	0.140832	0.623355	
2987	(11, 2, 6)	(5, 12, 4)	0.209246	0.464359	

	support	confidence	lift	leverage	conviction	zhangs_metric
3820	0.097695	0.997881	1.600824	0.036667	177.710981	0.416054
2817	0.106757	0.997813	1.600714	0.040064	172.185237	0.420241
3497	0.099439	0.997580	1.600341	0.037303	155.607321	0.416667
3887	0.109382	0.997579	2.148295	0.058466	221.292863	0.600340
4011	0.099251	0.997575	1.600333	0.037232	155.313538	0.416576
3985	0.104687	0.997540	2.148210	0.055955	217.740656	0.597166
3839	0.091774	0.997535	2.148198	0.049053	217.282289	0.588650
3443	0.127555	0.997472	1.600168	0.047842	148.974538	0.430061

```
2227  0.140463    0.997379  1.600019  0.052675  143.691489        0.436477
2987  0.208691    0.997348  2.147796  0.111526  201.971671        0.675819
```

6 Apriori (activate)

[35]:
```
ids_to_remove = ['12','15','29']
ap_act_df_00 = ap_act_df_00.drop(columns=ids_to_remove)
ap_act_df_11 = ap_act_df_11.drop(columns=ids_to_remove)
```

[36]:
```
# APRIORI ALGORITHM (group 0)

ap_set = ap(ap_act_df_00, min_support=supp_thresh, use_colnames=True)
rules = ap_rl(ap_set, metric = "confidence", min_threshold= lift_thresh)

# vectorize length func
v_len = np.vectorize(len)

# use it for condition in rule filtering
cond_0h = v_len(rules.antecedents.values) >= n_item_thresh
cond_0t = v_len(rules.consequents.values) >= n_item_thresh

# Apply both conditions to filter rules
filtered_rules = rules[cond_0h & cond_0t]

# Sort filtered rules by multiple columns for more insightful results
sorted_rules = filtered_rules.sort_values(by=filter_cols, ascending=[False,␣
 ↪False, False])

# Display top 10 meaningful and impactful rules
top_rules = sorted_rules.head(10)
top_rules
```

```
/opt/anaconda3/lib/python3.11/site-
packages/mlxtend/frequent_patterns/fpcommon.py:109: DeprecationWarning:
DataFrames with non-bool types result in worse computationalperformance and
their support might be discontinued in the future.Please use a DataFrame with
bool type
  warnings.warn(
```

[36]:
```
                     antecedents consequents  antecedent support  \
134496   (19, 24, 11, 7, 3, 1, 8)   (5, 6, 4)            0.095570
135439  (19, 24, 11, 27, 7, 3, 1)   (5, 6, 4)            0.090435
111396      (19, 24, 11, 7, 3, 1)   (5, 6, 4)            0.103380
110828       (24, 11, 7, 3, 1, 8)   (5, 6, 4)            0.100686
111820      (19, 24, 11, 3, 1, 8)   (5, 6, 4)            0.095949
111544      (24, 11, 27, 7, 3, 1)   (5, 6, 4)            0.095371
134864   (24, 11, 27, 7, 3, 1, 8)   (5, 6, 4)            0.094584
```

```
114995      (19, 24, 11, 7, 1, 8)   (5, 6, 4)            0.113152
111575      (24, 11, 28, 7, 3, 1)   (5, 6, 4)            0.091393
74788           (24, 11, 7, 3, 1)   (5, 6, 4)            0.108695

        consequent support   support  confidence      lift  leverage  \
134496            0.930628  0.095533    0.999622  1.074137  0.006594
135439            0.930628  0.090399    0.999600  1.074114  0.006237
111396            0.930628  0.103335    0.999563  1.074073  0.007126
110828            0.930628  0.100641    0.999551  1.074061  0.006940
111820            0.930628  0.095904    0.999529  1.074037  0.006611
111544            0.930628  0.095325    0.999526  1.074034  0.006571
134864            0.930628  0.094539    0.999522  1.074030  0.006516
114995            0.930628  0.113098    0.999521  1.074028  0.007795
111575            0.930628  0.091348    0.999505  1.074012  0.006295
74788             0.930628  0.108641    0.999501  1.074007  0.007486

        conviction  zhangs_metric
134496  183.350196       0.076313
135439  173.499372       0.075860
111396  158.667638       0.076917
110828  154.533067       0.076674
111820  147.262881       0.076249
111544  146.374920       0.076198
134864  145.167847       0.076128
114995  144.721554       0.077720
111575  140.270184       0.075843
74788   139.021488       0.077311
```

[37]:
```python
# APRIORI ALGORITHM (group 1)

ap_set = ap(ap_act_df_11, min_support=supp_thresh, use_colnames=True)
rules = ap_rl(ap_set, metric = "confidence", min_threshold= lift_thresh)

# vectorize length func
v_len = np.vectorize(len)

# use it for condition in rule filtering
cond_1h = v_len(rules.antecedents.values) >= n_item_thresh
cond_1t = v_len(rules.consequents.values) >= n_item_thresh

# Apply both conditions to filter rules
filtered_rules = rules[cond_1h & cond_1t]

# Sort filtered rules by multiple columns for more insightful results
sorted_rules = filtered_rules.sort_values(by=filter_cols, ascending=[False,␣
 ↪False, False])
```

```
# Display top 10 meaningful and impactful rules
top_rules = sorted_rules.head(10)
top_rules
```

```
/opt/anaconda3/lib/python3.11/site-
packages/mlxtend/frequent_patterns/fpcommon.py:109: DeprecationWarning:
DataFrames with non-bool types result in worse computationalperformance and
their support might be discontinued in the future.Please use a DataFrame with
bool type
  warnings.warn(
```

[37]:

	antecedents	consequents	antecedent support	consequent support	\
1142	(11, 2, 21, 6)	(5, 1, 4)	0.099069	0.439014	
1025	(11, 21, 6)	(5, 1, 4)	0.163654	0.439014	
1040	(11, 24, 6)	(5, 1, 4)	0.111186	0.439014	
889	(2, 21, 6)	(5, 1, 4)	0.104886	0.439014	
1101	(11, 3, 6)	(5, 19, 4)	0.170339	0.254819	
873	(11, 2, 6)	(5, 1, 4)	0.205228	0.439014	
1168	(11, 1, 3, 6)	(5, 19, 4)	0.162323	0.254819	
1141	(5, 2, 21, 6)	(11, 1, 4)	0.103015	0.464408	
1140	(6, 2, 21, 4)	(5, 11, 1)	0.103152	0.422290	
1026	(5, 21, 6)	(11, 1, 4)	0.170361	0.464408	

	support	confidence	lift	leverage	conviction	zhangs_metric
1142	0.098686	0.996135	2.269030	0.055193	145.141856	0.620784
1025	0.162910	0.995455	2.267481	0.091064	123.418868	0.668362
1040	0.109110	0.981333	2.235313	0.060298	30.052000	0.621767
889	0.101925	0.971771	2.213533	0.055879	19.872508	0.612473
1101	0.163932	0.962387	3.776750	0.120527	19.811554	0.886172
873	0.197345	0.961591	2.190345	0.107248	14.605610	0.683782
1168	0.155962	0.960814	3.770580	0.114599	19.016695	0.877174
1141	0.098686	0.957976	2.062792	0.050845	12.744988	0.574391
1140	0.098686	0.956708	2.265524	0.055126	13.344443	0.622849
1026	0.162910	0.956264	2.059105	0.083793	12.246045	0.619971